

Testes de Software

Profº Renato de Souza Garcia

Sumário

- Testes de Integração
- Motivação
- Diferenças entre Teste Unitário e de Integração
- Principais Ferramentas JS para testes de integração
- Exemplos de Testes de casos de testes para teste de integração
- Criando um projeto de API REST

O que são testes de integração?

- São testes realizados para validar se os módulos ou componentes funcionam juntos.
- Testam a comunicação (interface) entre partes do sistema.
- Após os testes unitários é que são realizados os testes de integração

Motivação

- Realizar testes de integração serve para:
 - Evitar erros na troca de dados entre módulos.
 - Assegurar que APIs, banco de dados e serviços externos se comunicam corretamente.
 - Reduzir bugs que aparecem na integração.

Diferença entre Teste Unitário e de Integração

Teste Unitário	Teste de Integração
Testa funções isoladas	Testa se funções/módulos funcionam juntos
Utiliza de mocks e stubs	Usa sistemas reais ou parcialmente reais
Mais rapido / focado	Mais abrangente e próximo da realidade

Principais Ferramentas JS para testes de integração

- SuperTest
 - Foco: Testes de integração para APIs HTTP
 - Permite testar rotas HTTP de servidores Express (ou qualquer backend Node.js)
 - Fácil integração com Jest, Mocha, Vitest e outros
 - Leve e simples de configurar.
 - Mais usada para testar APIs REST.

Principais Ferramentas JS para testes de integração

- Jest (com suporte a integração)
 - Foco: Originalmente testes unitários, mas suporta integração
 - Possui recursos integrados como mocks, spies e testes assíncronos
 - Muito usado junto com SuperTest para testar integração de APIs
 - Integra cobertura de código (Coverage)

Principais Ferramentas JS para testes de integração

- Mocha + Chai + SuperTest
 - Mocha: Framework de teste (estrutura de describe/it)
 - Chai: Biblioteca de asserções (expect, should)
 - SuperTest: Para requisições HTTP
 - Conjunto clássico, mais configurável e modular que Jest

Supertest

- Para essa disciplina utilizamos o supertest para realizar os testes de integração
- é uma ferramenta bastante popular e consolidada para realizar testes de integração utilizando JavaScript

Exemplos de casos de testes para teste de integração

- Testar uma API - verificando se responde corretamente
- Verificar se um POST cria um registro
- Verificar se um endpoint retorna os dados esperados
- Simular sequências de requisições
- Validar o fluxo completo da requisição:
cliente de teste → API → rota → regra de negócio →
resposta HTTP

Por que esse é um teste de integração?

- No teste de integração, não chamamos uma função isolada diretamente.
- Com o SuperTest, enviamos uma requisição HTTP para a aplicação Express e validamos a resposta gerada pela integração entre:
 - app.js
 - middlewares
 - arquivo de rotas
 - regra de criação/listagem
 - resposta HTTP em JSON

Criando um projeto de API REST

- Vamos criar um projeto simples para armazenar usuários
- Crie um diretório **api-teste-integracao**
- execute o comando: **npm init -y**
- instale o express: **npm install express**
- instale o jest e o supertest:
npm install --save-dev jest supertest

Criando um projeto de API REST

- A estrutura do nosso projeto deve ser:

```
✓ API-TESTE-INTEGRACAO
  ✓ data
    JS usersData.js
  > node_modules
  ✓ routes
    JS users.js
  ✓ tests
    JS users.test.js
  JS app.js
  {} package-lock.json
  {} package.json
  JS server.js
```

Criando um projeto de API REST

- Criando o arquivo de dados **usersData.js** (rotas)

```
const initialUsers = [  
  { id: 1, name: 'Aline Dias' },  
  { id: 2, name: 'Roberto Carlos' }  
];  
  
const users = [...initialUsers];  
  
function resetUsers() {  
  users.length = 0;  
  users.push(...initialUsers);  
}  
  
module.exports = {  
  ...  
  users,  
  resetUsers  
};
```

Criando um projeto de API REST

- Criando o arquivo **users.js** (rotas)

```
const express = require('express');
const router = express.Router();
const { users } = require('../data/usersData');

router.get('/', (req, res) => {
  res.json(users);
});

router.post('/', (req, res) => {
  const { name } = req.body;

  if (!name) {
    return res.status(400).json({
      error: 'É necessário preencher o nome.'
    });
  }

  const newUser = {
    id: users.length + 1,
    name
  };

  users.push(newUser);

  res.status(201).json(newUser);
});

module.exports = router;
```

Criando um projeto de API REST

- Configurando o módulo **app.js** para exportar e enviar ao servidor

```
const express = require('express')
const app = express();
const usersRouter = require('./routes/users');

app.use(express.json());
app.use('/users', usersRouter);

module.exports = app;
```


Criando um projeto de API REST

- Configurando o módulo **server.js** para exportar e enviar ao servidor

```
const app = require('./app');  
  
const PORT = 3000;  
  
app.listen(PORT, () => {  
  console.log(`Servidor executando na porta ${PORT}`);  
});
```

Criando um projeto de API REST

- Escrevendo os testes em **users.test.js**
- O código testa GET /users, POST /users e erro no POST
- Uso do beforeEach para garantir independência entre os testes.

```
const request = require('supertest');
const app = require('../app');

const { resetUsers } = require('../data/usersData');

describe('Testes de integração na API /users', () => {
  beforeEach(() => {
    resetUsers();
  });

  it('GET /users deve retornar lista de usuários', async () => {
    const res = await request(app).get('/users');

    expect(res.statusCode).toBe(200);
    expect(res.body).toBeInstanceOf(Array);
    expect(res.body.length).toBe(2);
  });

  it('POST /users deve criar um novo usuário', async () => {
    const res = await request(app)
      .post('/users')
      .send({ name: 'Charlie' });

    expect(res.statusCode).toBe(201);
    expect(res.body).toHaveProperty('id');
    expect(res.body.name).toBe('Charlie');
  });

  it('POST /users sem name deve retornar erro 400', async () => {
    const res = await request(app)
      .post('/users')
      .send({});

    expect(res.statusCode).toBe(400);
    expect(res.body).toHaveProperty(
      'error',
      'É necessário preencher o nome.'
    );
  });
});
```

Criando um projeto de API REST

- Nosso **package.json** ficará da seguinte forma:

```
{
  "name": "api-teste-integracao",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  ▷ Debug
  "scripts": {
    "start": "node server.js",
    "test": "jest"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1"
  },
  "devDependencies": {
    "jest": "^30.4.2",
    "supertest": "^7.2.2"
  }
}
```

Criando um projeto de API REST

- Após terminar de criar e definir os elementos do projeto, execute os testes:
`npm test`

Para praticar...

- Codifique os seguintes endpoints:
 - GET /users/:id
 - PUT /users/:id
 - DELETE /users/:id
- Em seguida, crie testes de integração para validar:
 - busca de usuário existente;
 - erro 404 para usuário inexistente;
 - atualização de usuário;
 - remoção de usuário;
 - tentativa de remover usuário inexistente.

Evoluindo o Projeto - adicionando a entidade tarefas

- Criaremos uma nova entidade chamada tarefas.
- Uma tarefa deve conter:
 - id
 - title
 - completed
 - userId

Evoluindo o Projeto - adicionando a entidade tarefas

- criando o arquivo de dados **taskData.js**

```
const initialTasks = [
  {
    id: 1,
    title: 'Estudar testes de integração',
    completed: false,
    userId: 1
  },
  {
    id: 2,
    title: 'Criar testes com SuperTest',
    completed: true,
    userId: 2
  }
];

const tasks = [...initialTasks];

function resetTasks() {
  tasks.length = 0;
  tasks.push(...initialTasks);
}

module.exports = {
  tasks,
  resetTasks
};
```

Evoluindo o Projeto - adicionando a entidade tarefas

- criando o arquivo de rotas tasks.js

```
const express = require('express');
const router = express.Router();

const { tasks } = require('../data/tasksData');
const { users } = require('../data/usersData');

router.get('/', (req, res) => {
  res.json(tasks);
});

router.post('/', (req, res) => {
  const { title, userId } = req.body;

  if (!title) {
    return res.status(400).json({
      error: 'É necessário informar o título da tarefa.'
    });
  }

  const userExists = users.find(user => user.id === userId);

  if (!userExists) {
    return res.status(404).json({
      error: 'Usuário não encontrado.'
    });
  }

  const newTask = {
    id: tasks.length + 1,
    title,
    completed: false,
    userId
  };

  tasks.push(newTask);

  res.status(201).json(newTask);
});

module.exports = router;
```


Evoluindo o Projeto - adicionando a entidade tarefas

- alterando o arquivo de rotas users.js
- importar tasksData.js
- inclua o método para listar tasks por user

```
const { tasks } = require('../data/tasksData');
```

```
router.get('/:id/tasks', (req, res) => {  
  const id = Number(req.params.id);  
  
  const user = users.find(user => user.id === id);  
  
  if (!user) {  
    return res.status(404).json({  
      error: 'Usuário não encontrado.'  
    });  
  }  
  
  const userTasks = tasks.filter(task => task.userId === id);  
  
  res.json(userTasks);  
});
```

Evoluindo o Projeto - adicionando a entidade tarefas

- alterando o arquivo app.js

```
const express = require('express');
const app = express();

const usersRouter = require('./routes/users');
const tasksRouter = require('./routes/tasks');

app.use(express.json());

app.use('/users', usersRouter);
app.use('/tasks', tasksRouter);

module.exports = app;
```

Evoluindo o Projeto - adicionando a entidade tarefas

- criando o arquivo de testes tasks.test.js

```
it('POST /tasks deve retornar erro se o usuário não existir', async () => {
  const res = await request(app)
    .post('/tasks')
    .send({
      title: 'Tarefa inválida',
      userId: 999
    });

  expect(res.statusCode).toBe(404);
  expect(res.body).toHaveProperty(
    'error',
    'Usuário não encontrado.'
  );
});

it('GET /users/:id/tasks deve retornar tarefas do usuário', async () => {
  const res = await request(app).get('/users/1/tasks');

  expect(res.statusCode).toBe(200);
  expect(res.body).toBeInstanceOf(Array);

  res.body.forEach(task => {
    expect(task.userId).toBe(1);
  });
});
```

Evoluindo o Projeto

- adicionando a entidade tarefas

- incluindo testes que validam a integração entre as entidades em tasks.test.js
- Apesar de ser uma rota em users, os últimos testes devem ficar em tarefas pela sua relação direta

```
it('POST /tasks deve retornar erro se o usuário não existir', async () => {
  const res = await request(app)
    .post('/tasks')
    .send({
      title: 'Tarefa inválida',
      userId: 999
    });

  expect(res.statusCode).toBe(404);
  expect(res.body).toHaveProperty(
    'error',
    'Usuário não encontrado.'
  );
});

it('GET /users/:id/tasks deve retornar tarefas do usuário', async () => {
  const res = await request(app).get('/users/1/tasks');

  expect(res.statusCode).toBe(200);
  expect(res.body).toBeInstanceOf(Array);

  res.body.forEach(task => {
    expect(task.userId).toBe(1);
  });
});

it('GET /users/:id/tasks deve retornar tarefas do usuário', async () => {
  const res = await request(app).get('/users/1/tasks');

  expect(res.statusCode).toBe(200);
  expect(res.body).toBeInstanceOf(Array);

  res.body.forEach(task => {
    expect(task.userId).toBe(1);
  });
});
```

Executando os Testes

- rode o comando
npm test

```
● rgarcia@MacBook-Air-de-Renato api-teste-integracao % npm test

> api-teste-integracao@1.0.0 test
> jest

PASS tests/users.test.js
PASS tests/tasks.test.js

Test Suites: 2 passed, 2 total
Tests:       8 passed, 8 total
Snapshots:   0 total
Time:        0.264 s
Ran all test suites.
○ rgarcia@MacBook-Air-de-Renato api-teste-integracao %
```

Incluindo novos scripts

- Modifique o package.json para trazer a cobertura de testes:

```
"scripts": {  
  "start": "node server.js",  
  "test": "jest",  
  "test:coverage": "jest --coverage"  
},
```


Boas práticas em testes de integração

- Ter um banco de dados de teste separado
- Planejar e definir conjunto de dados conhecidos e controlados para testar.
- Testes independentes (não dependem da ordem de execução)
- Limpar os dados antes e depois dos testes.
 - **Antes dos testes:** Garantir que o banco ou a estrutura de dados esteja em um estado inicial (vazio ou conhecido).
 - **Depois dos testes:** Remover dados criados durante os testes para não afetar outros testes ou o ambiente.

O que foi feito nessa aula...

- Conhecemos o que é teste de integração
- Configuramos o ambiente de testes de integração
- Criamos um projeto de testes com SuperTest para endpoints da API
- Verificamos respostas, status e dados retornados

Atividade Prática

- Agora, crie uma nova entidade para ampliar a API.
- Escolha uma entidade que possua correlação com usuários e tarefas.

Fim